

Introduction & goals

- **What is an OS?**
 - Program that acts as an **intermediary** between users (and applications) and hardware.
 - Provides a user-friendly environment to develop & run programs (so apps don't need direct hardware knowledge).
- **Primary goals**
 - Execute user programs and simplify problem solving.
 - Make the system convenient and efficient.
 - Manage resources: CPU(s), memory, I/O devices.
- **Two views**
 - **Resource allocator** — decides who gets what.
 - **Control program** — controls execution to prevent errors and misuse.

Computer system organization (concept + diagram both arte imp)

- **Components**
 - CPUs, main memory, device controllers (disk, USB, graphics), I/O devices.
 - Device controllers have **local buffers** and registers.
- **How they interact**
 - CPU $\leftrightarrow$ memory $\leftrightarrow$ device controllers (shared bus).
 - Device controllers perform transfers to their buffers; CPU moves data to/from main memory.
- **Diagram to study:** *"A modern computer system"* (page ~15)
 - Explains CPU, disk controller, USB controller, graphics adapter all connected to memory via bus.
 - **Explanation:** shows concurrency: CPU and device controllers can run concurrently, competing for memory cycles; memory controller coordinates access.

Bootstrapping & interrupts

- **Bootstrap (firmware):** ROM/EEPROM code that initializes hardware and loads the OS kernel.
- **Interrupts**
 - OS is **interrupt-driven**: devices signal completion by interrupts.
 - **Trap (exception)** = software-generated interrupt (e.g., division by zero or system call).
 - **Interrupt handling flow**: save CPU state → identify interrupt → call service routine → restore state.
- **Diagram to study:** *Interrupt timeline & vectored interrupt images (pages ~18–20)*
 - Shows CPU being preempted, I/O device transferring, and how interrupts cause context switches.

Storage structure & caching diagram below

- **Main memory**
 - Volatile, CPU-accessible (DRAM).
- **Secondary storage**
 - Hard disks (tracks/sectors), SSDs (flash) — nonvolatile, large capacity.
- **Hierarchy (diagram on page ~25)**
 - Registers → cache → main memory → SSD → HDD → optical → magnetic tape.
 - **Principles**: faster = more expensive, smaller; caching copies hot data to faster storage for speed.
- **Caching**
 - Check cache first; if miss → fetch from slower storage and update cache.
 - Cache management (size, replacement policy) is critical.

I/O structure & Direct Memory Access (DMA)

- **Device driver + controller**
 - Driver gives a uniform interface; controller handles hardware specifics.
- **Programmed I/O vs DMA**

- Programmed I/O: CPU moves each byte — inefficient.
- **DMA:** controller transfers whole blocks to memory with minimal CPU involvement. Only one interrupt per block (much fewer interrupts).
- **Diagram to study:** *DMA structure (page ~30)* — device controller ↔ memory (DMA), CPU interrupted per block.

OS services & system calls

- **High-level services**
 - User interface (CLI, GUI), program execution, I/O operations, file system, communications, error detection, resource allocation, accounting, protection/security.
- **System calls**
 - Interface for programs to request OS services (often wrapped by C library functions).
- **Types of system calls (6 groups)**
 1. Process control
 2. File manipulation
 3. Device manipulation
 4. Information maintenance
 5. Communications
 6. Protection
- **Example flow:** `printf()` → C library → `write()` system call → kernel handles I/O → returns to user.

Process concept (memory layout + PCB)

- **Process vs program**
 - Program = passive code on disk. Process = program **in execution** (active).
- **Process memory layout**
 - **Text** (code) | **Data** (globals) | **Heap** (dynamic alloc) | **Stack** (local variables, return addresses).

- Diagram(s) show stack growing downward, heap upward (typical layout).
- **Process Control Block (PCB)**
 - OS representation of a process; contains:
 - Process state (new, ready, running, waiting, terminated)
 - Program counter
 - CPU registers
 - CPU scheduling info (priority, queues)
 - Memory-management info
 - Accounting info
 - I/O status (open files, devices)

Process states (simple finite-state model)

- **States**
 - New → Ready → Running → Waiting (blocked) → Ready → Running → Terminated
- **State transitions**
 - **I/O or event wait** → Running → Waiting.
 - **I/O completion** → Waiting → Ready.
 - **Scheduler preemption or yield** → Running → Ready.

Process creation & termination (brief)

- **Creation**
 - Parent creates child (fork in UNIX); child may call exec to load new program.
 - Resource-sharing options: share all, share subset, share none.
- **Termination**
 - exit() or abnormal (signals). Parent collects status via wait() / waitpid().
 - Zombies: child terminated but parent hasn't waited.
 - Orphans: parent terminated before child — reparented (often to init).

Context switch

- **What happens**
 - Save current PCB (registers, PC, etc.) → load new PCB → update memory mappings → start new process.
- **Notes**
 - Switch time = overhead; reduce frequency or make it fast.
 - More complex OS/PCB → longer context switch.

I/O request life cycle (concise)

- **Steps**
 1. Process issues system call (read/write/open).
 2. Kernel validates and enqueues I/O request in device queue.
 3. Device driver converts request to hardware commands and starts controller.
 4. Process typically blocks (waiting state); scheduler runs others.
 5. Device completes I/O → sends interrupt.
 6. ISR handles completion; kernel moves process to Ready.
 7. Scheduler runs process; system call returns to user mode.
- **Diagram to study:** *Life cycle / I/O queue images (pages ~121–124).*

CPU scheduling — basics & criteria

- **Why scheduling?**
 - Only one process per CPU core at a time (unless multicore). Multiprogramming keeps CPU busy.
- **Scheduling metrics**
 - CPU utilization, throughput, turnaround time, waiting time, response time.
- **Schedulers**
 - **Long-term** (job scheduler) — degree of multiprogramming (infrequent).
 - **Short-term** (CPU scheduler) — chooses next process (very frequent).
 - **Medium-term** — swapping to control degree of multiprogramming.

Common scheduling algorithms (with key properties & diagram notes)

- **FCFS (First-Come, First-Served)** — non-preemptive
 - Simple; may cause **convoy effect** (short tasks wait behind long ones).
 - Not great for interactive/time-sharing systems.
 - Example on slides (pages ~140–142) — shows Gantt charts and variable average wait times.
- **SJF (Shortest Job First)** — non-preemptive (optimal for average waiting time)
 - Needs knowledge (or estimate) of next CPU burst.
 - **Preemptive version:** Shortest Remaining Time First (SRTF).
 - **Burst prediction:** exponential averaging:
 - $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$
 - α in $[0,1]$ weights recent history (slides show worked example).
- **Priority scheduling**
 - Each process has priority; highest priority runs first.
 - Can be **preemptive** or **non-preemptive**.
 - **Starvation** for low-priority processes; solution: **aging** (increase priority over time).
- **Round-Robin (RR)**
 - Designed for time-sharing. Ready queue is circular; each process gets a time quantum q .
 - Preemptive; good response time if q is small, but too small $q \rightarrow$ many context switches (overhead).
 - Example & performance notes on pages ~156–158.
- **Choosing q (RR trade-offs)**
 - If q too large $\rightarrow$ behaves like FCFS.
 - If q too small $\rightarrow$ high dispatcher overhead.

Practical tips & formulas to remember

- **Exponential averaging** for burst prediction (use $\alpha \sim 0.5$ as common default).
- **Compute waiting time** from Gantt charts (total waiting / #processes).
- **Context-switch cost**: try to balance between fairness (preemption) & overhead.
- **Use RR for interactivity**, SJF for minimizing average waiting time (if bursts known).

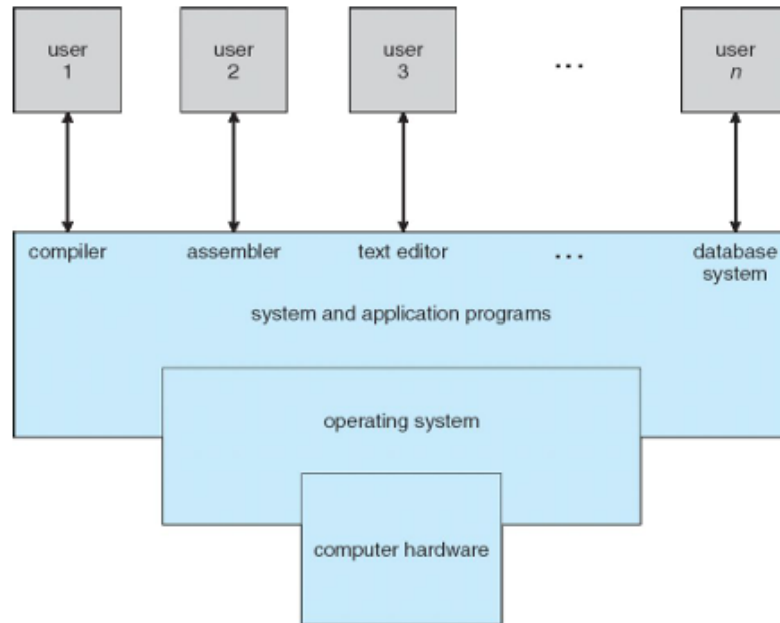
Diagrams to review in the PDF (quick list with page references)

- Need for OS

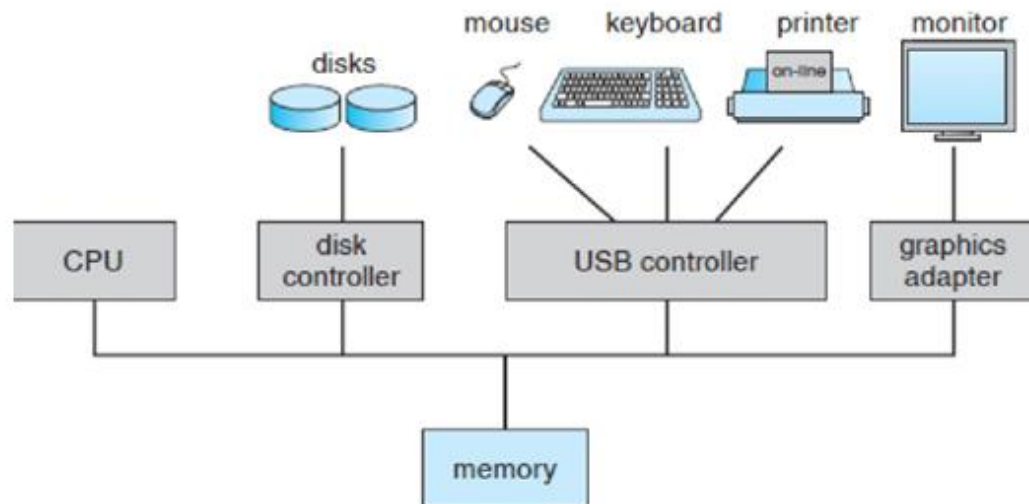


- Four components (user / system / OS / hardware):

Four Components of a Computer System (Abstract view)



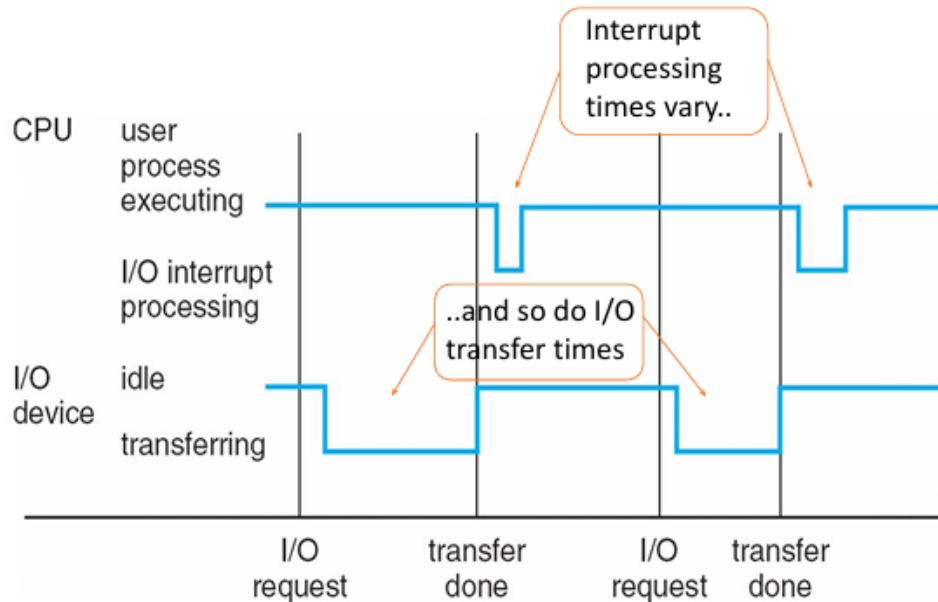
- Modern computer system (bus, controllers):



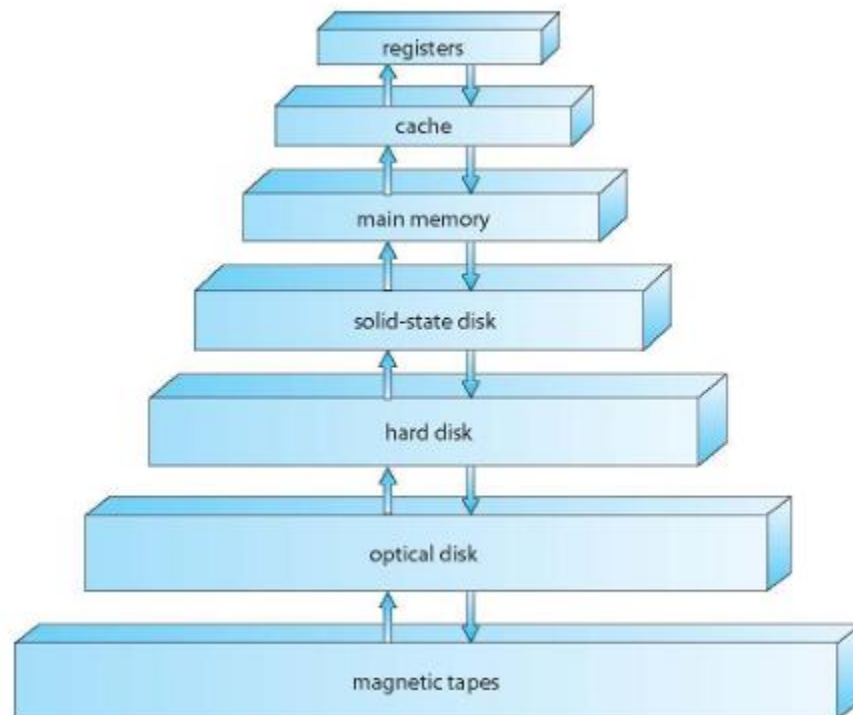
A modern computer system

- Interrupt timeline (CPU / I/O / device):

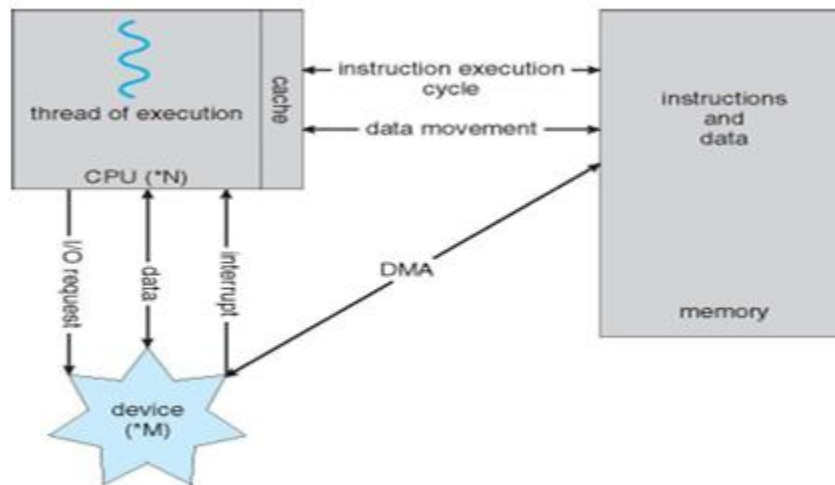
Interrupt Timeline for a single process doing output



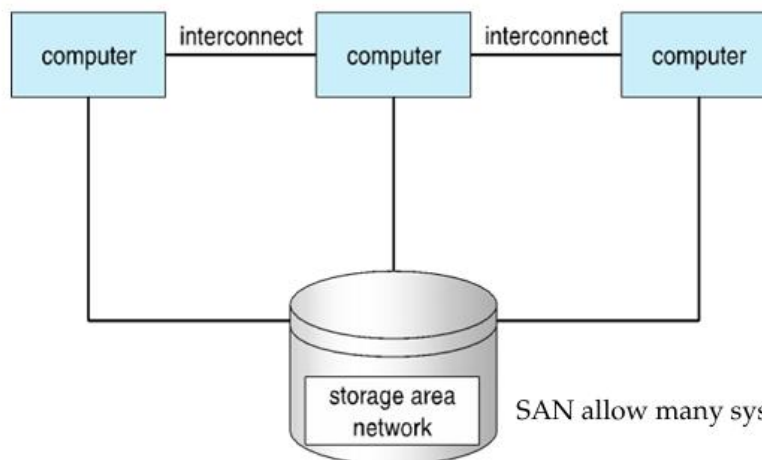
- Storage-device hierarchy (registers → tape):



- DMA structure:

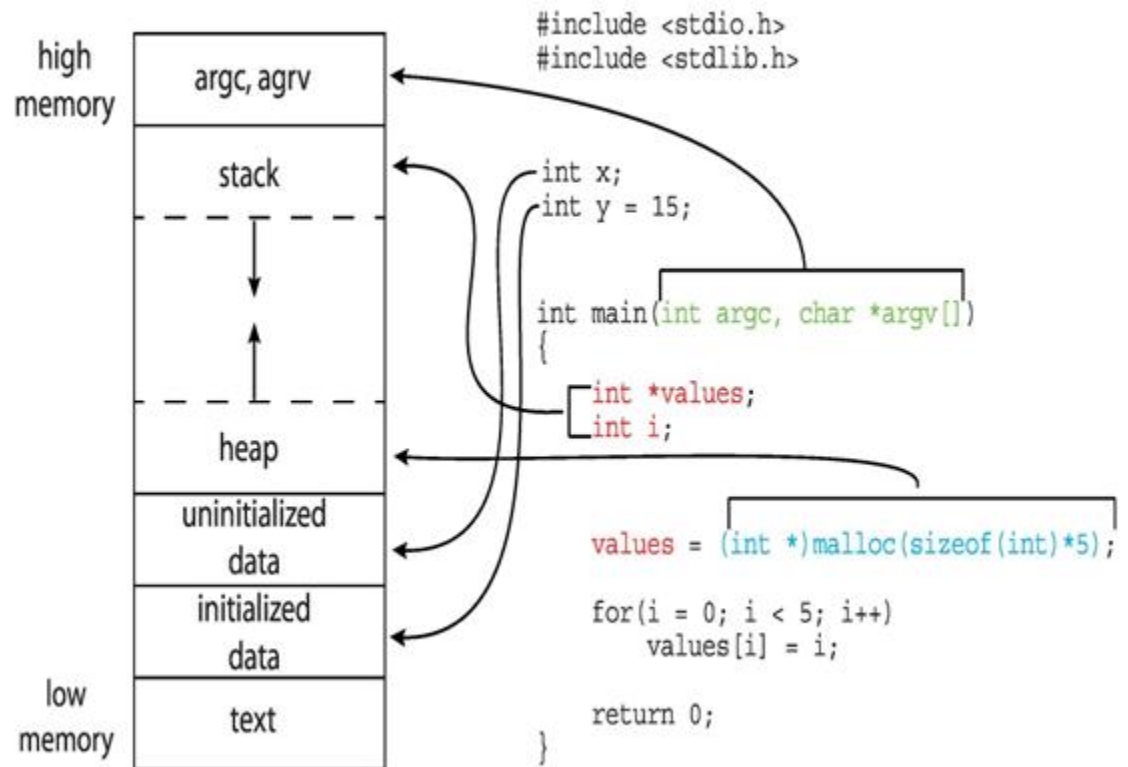
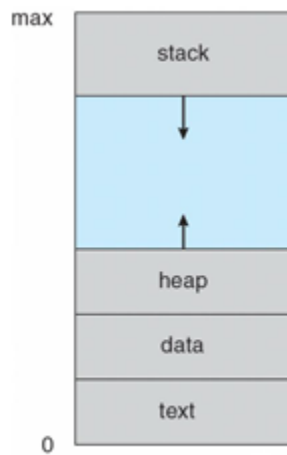


- Clustered system & SAN:

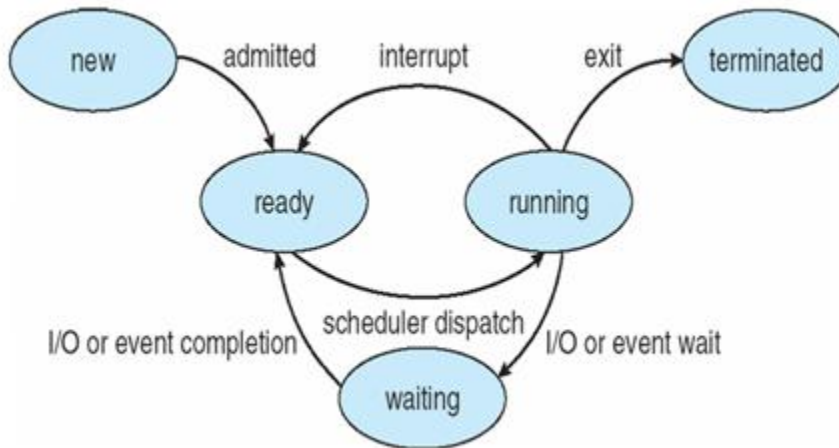


General structure of a clustered system.

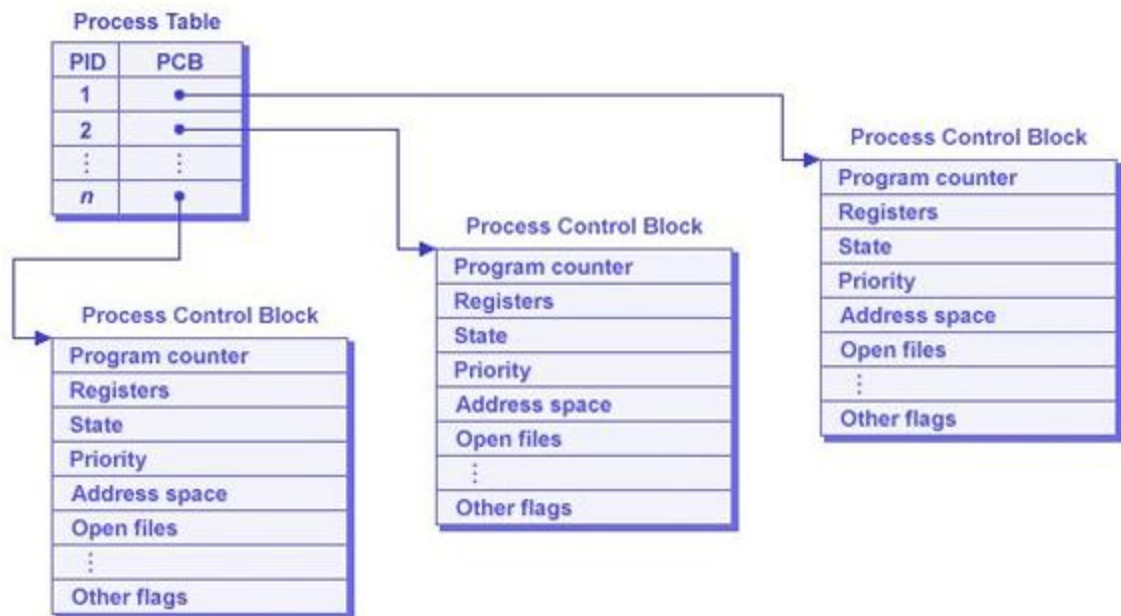
- Memory layout of process (text/data/heap/stack):



process states

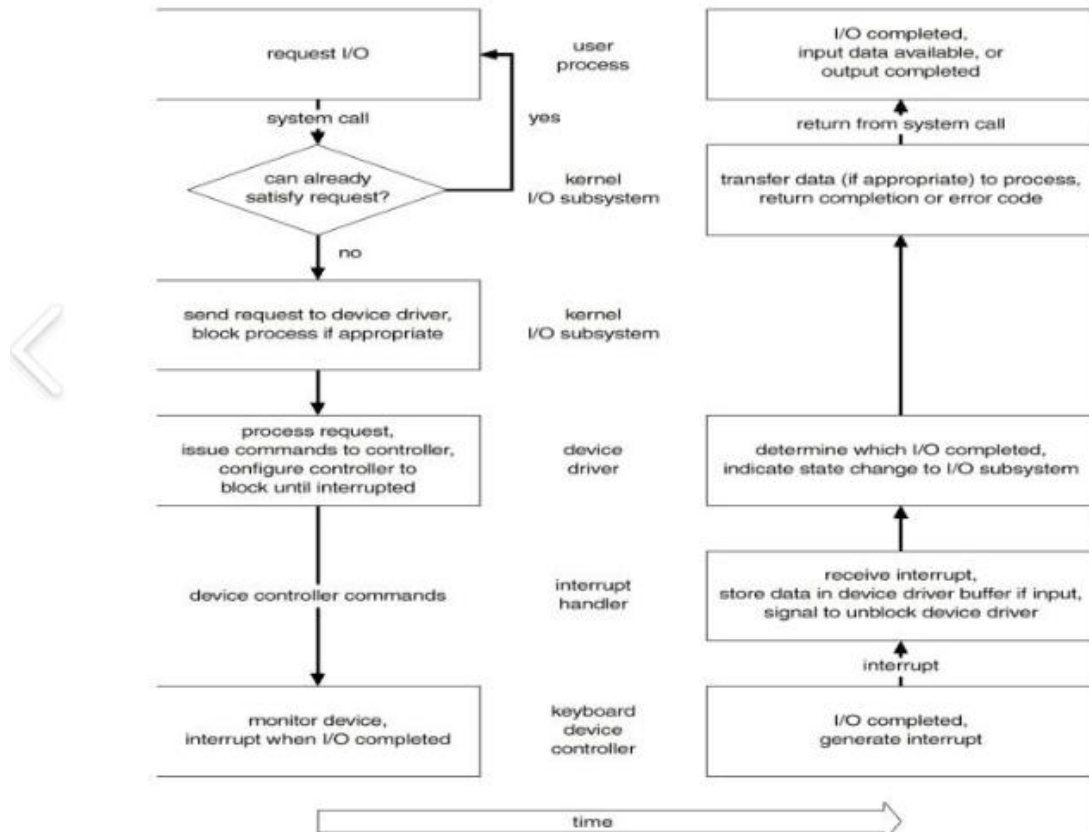


- PCB contents (box list): *page ~95*



- I/O life cycle diagrams: *pages*

Lifecycle of I/O Request



- **PCB** = the OS's per-process data (state, PC, registers, scheduling info, memory info, I/O).
- **Trap** = software interrupt. **Interrupt** = hardware event.
- **DMA** = fast block transfer without CPU per-byte involvement.
- **Scheduler types**: long-term, medium-term, short-term.
- **SJF optimality**: SJF minimizes average waiting time if burst lengths are known.